

Structs

Tuesday, 4 August 2026

10:02 AM

+ Structs in Go-:

We use structs in Go to represent structured data. It's often convenient to group different type of variables together.

e.g.

```
type car struct {  
    brand string  
    model string  
    y doors int  
}
```

* Nested structs in Go-:

Structs can be nested to represent more complex entities:

* Anonymous structs in Go-:

An anonymous struct is just like a normal struct, but it is defined without a name and therefore cannot be referenced elsewhere in the code.

e.g.

```
type car struct {  
    brand string  
    doors int  
    wheel struct {  
        radius int  
    }  
}
```

* Embedded structs-:

Go is not an object-oriented language. However, embedded structs provide a kind of data-only inheritance that can be useful at times. Keep in mind, Go doesn't support classes or inheritance in the complete sense, but embedded structs are a way to elevate and share fields between struct definitions.

- Embedded vs Nested-:

- Unlike nested structs, an embedded struct's fields are accessed at the top level like normal fields
- Like nested structs you assign the promoted fields with the embedded struct in a composite literal.

+ Structs Methods in Go-:

While Go is not object-oriented, it does support methods that can be defined on structs. Methods are just functions that have a receiver as a special parameter that syntactically goes before the name of the function.

+ Memory Layout-:

In Go, structs sit in memory in contiguous block, with fields placed one after another as defined in the struct.

e.g.

```
type states struct {  
    Reach uint16  
    NumPosts uint8  
    NumLikes uint16  
}
```

Reach uint16 (2 bytes)	NumPosts uint8 (1 byte)	NumLikes uint16 (2 bytes)
------------------------------	-------------------------------	---------------------------------

states struct (4 byte total)

- the order of fields in a struct can have a big impact on memory usage.

+ Empty struct-:

Empty struct are used in Go as a unary value. Empty struct are Go's smallest possible type: they take up zero byte of memory.

e.g.

```
empty := struct{}{}
```